

# Klipper — Wealth Operating System

Sistema pessoal de gestão financeira: contas, transações, orçamento, investimentos e um copiloto de IA (Kira). Este documento consolida arquitetura, decisões técnicas, inventário de telas, sistema de design e padrões de interação para handoff com o coordenador de design.

Repo · [github.com/Borussia80/Klipper](https://github.com/Borussia80/Klipper)   Monorepo · 3 apps   Telas ativas · 11

Stack atual · Nuxt 4 + Rails 8

01 — visão geral

## Arquitetura

Klipper é um **monorepo** com três aplicações independentes. O produto principal é um PWA (app instalável, funciona offline-first) servido separadamente da API de domínio — não há SSR acoplado ao backend de negócio.

● **klipper-web**

Nuxt 4 (Vue 3) · PWA · frontend do app financeiro. 11 rotas ativas, componentes em Vue SFC.

● **klipper-api**

Rails 8 API-only · PostgreSQL · JWT auth · domínio de contas, transações, orçamento e investimentos.

● **quebec-web**

Nuxt 4 · landing institucional da Quebec (empresa-mãe do produto), separada do app logado.

### Stack por camada

| CAMADA   | ESCOLHA                                                                                                  | DEPLOY |
|----------|----------------------------------------------------------------------------------------------------------|--------|
| Frontend | Nuxt 4 · Vue 3 · TypeScript · PWA (Serwist)                                                              | Vercel |
| Gráficos | React 18 + Recharts, montado como "ilha" isolada dentro do Nuxt via <a href="#">@vitejs/plugin-react</a> | —      |
| Backend  | Ruby on Rails 8 (API-only) · Solid Queue · Solid Cache                                                   | Render |
| Banco    | PostgreSQL                                                                                               | —      |

Auth      JWT (JwtService) + bcrypt      —

State (front)      Composables Vue (sem Vuex/Pinia store global — dados vêm via `useAsyncData` /composables por domínio)      —

Decisão notável para o design: os três gráficos do produto (sparkline de patrimônio, timeline de patrimônio, donut de alocação) rodam em **React embarcado**, não em Vue nativo — qualquer alteração visual nesses componentes cruza um limite técnico (props tipadas, sem CSS vars diretas — usam tokens de gráfico próprios, ver `design-system` ).

## 02 – histórico

# Decisões de arquitetura

O produto passou por três reescritas de stack em poucos meses. Cada uma teve motivo específico — importante para não propor voltar a padrões já descartados.

até jun/2026

### Streamlit (Python)

Protótipo original. Multi-page app com `st.dialog` para modais.  
Descartado: limitações de UI/mobile e de customização de interação.

jun/2026

### Next.js 16 + React 19 (PWA) na Vercel

Primeira reescrita como PWA de verdade (Serwist, ícones, auth via Supabase). Descartada em favor de Vue/Nuxt por preferência de stack do time.

21/jun/2026 → atual

### Nuxt 4 (frontend) + Rails 8 (backend)

Decisão vigente. Frontend em Vue/Nuxt, domínio de negócio concentrado em Rails (auth, jobs, cache, filas). React mantido apenas como ilha isolada para os gráficos (Recharts).

06/jul/2026

### Deploy da API migra para Render

Configuração Kamal/Render substitui a tentativa anterior via Railway/Vercel serverless para a API.

# Mapa de telas

Navegação principal fica em sidebar fixa (desktop) com espelho em bottom-nav (mobile) — mesma lista de itens, dois padrões de exposição.

/dashboard

/transacoes

/orcamento

/investimentos

/contas

/relatorios

/kira

/importar · BETA

| ROTA           | FUNÇÃO                                                                                   | STATUS    |
|----------------|------------------------------------------------------------------------------------------|-----------|
| /dashboard     | Hero: timeline de patrimônio líquido, KPIs com delta mensal e sparkline, próximas contas |           |
| /transacoes    | Lista agrupada por data, ícones por categoria, lançamento rápido                         |           |
| /orcamento     | Orçamento estilo envelope (inspirado em Actual Budget) por categoria                     |           |
| /investimentos | Donut de alocação, cards mobile, dividend yield %                                        |           |
| /contas        | Contas bancárias e cartões cadastrados                                                   |           |
| /relatorios    | Fechamento mensal + evolução de patrimônio líquido                                       |           |
| /kira          | Chat com assistente financeiro de IA                                                     | beta      |
| /importar      | Importação de extrato (CSV) com auto-categorização por regras                            | beta      |
| /onboarding    | Fluxo de 3 passos pré-login: conectar banco, categorizar, revisar                        |           |
| /login         | Autenticação + recuperação de senha                                                      |           |
| /configuracoes | Página existe e funciona, mas <b>não está linkada</b> em sidebar, topbar ou mobile-nav   | gap de IA |

`/configuracoes` é acessível só por URL direta hoje — vale decidir com o coordenador onde ela deveria aparecer (menu do usuário na topbar é o candidato natural).

# Design system

Tema único, dark-first (não há light mode implementado no produto ainda). Paleta fria, construída para leitura de números financeiros por longos períodos — contraste é tratado como requisito, não afterthought: os tokens de texto já vêm anotados com a razão de contraste WCAG contra o fundo.

## Paleta de texto (tokens.css)

|                                                          |                                                         |                                                        |                                                                |
|----------------------------------------------------------|---------------------------------------------------------|--------------------------------------------------------|----------------------------------------------------------------|
| <div></div> <div>--t1<br/>#E5EEF8<br/>16.4:1 · AAA</div> | <div></div> <div>--t2<br/>#9BBFDA<br/>8.5:1 · AAA</div> | <div></div> <div>--t3<br/>#6B93AE<br/>5.1:1 · AA</div> | <div></div> <div>--t4<br/>#3D6478<br/>2.4:1 · decorativo</div> |
|----------------------------------------------------------|---------------------------------------------------------|--------------------------------------------------------|----------------------------------------------------------------|

## Paleta funcional

|                                                              |                                           |                                         |                                           |                                            |
|--------------------------------------------------------------|-------------------------------------------|-----------------------------------------|-------------------------------------------|--------------------------------------------|
| <div></div> <div>--bg<br/>#07121E</div>                      | <div></div> <div>--blue<br/>#2B7DF4</div> | <div></div> <div>--ok<br/>#0DB878</div> | <div></div> <div>--warn<br/>#E59010</div> | <div></div> <div>--alert<br/>#E83535</div> |
| <div></div> <div>--pur<br/>#7C5CF5 · usado<br/>na Kira</div> |                                           |                                         |                                           |                                            |

## Instituições financeiras (cores de marca)

Bancos recebem cor própria e reconhecível — Nubank `#8A05BE`, Itaú `#EC7000`, BTG `#0B2545` — mais cripto `#F4C030`. Novos bancos/ativos precisam de um token de cor equivalente antes de entrar em produção.

## Tipografia

Números e labels técnicos usam **monospace (JetBrains Mono)** com `tabular-nums` — visível em rótulos de step ("Passo 1 de 3"), badges BETA e valores monetários. Texto de interface usa a fonte do sistema. Não há uma família serifada no produto.

## Layout & responsividade

Três colunas no desktop: sidebar fixa ( `--nav: 228px` ) · conteúdo · painel direito contextual ( `--rp: 284px` ), muda de conteúdo conforme a página — patrimônio no Dashboard, resumo de categoria em Transações etc). Em mobile, sidebar e painel direito somem; surge bottom-nav fixo com os mesmos 6 itens principais. Raio de borda padrão é discreto ( `--r: 6px` ).

## 05 – padrão de interação

# Modais

Não são modais centrais — é um **drawer lateral** (slide-in pela direita, 420px, full-screen em mobile), compartilhando um único componente-base ( `BaseModal` ) com focus-trap, fechamento por `Esc` , `aria-modal` e devolução de foco ao elemento que abriu o painel. Todas as 4 telas de criação usam esse mesmo padrão — decisão deliberada de consistência, não uma coincidência de implementação.

## Os 4 modais existentes

### Novo lançamento

Tipo:  
receita/despesa  
Valor (R\$)  
Descrição  
Categoria ·  
Conta

### Novo lançamento

O modal mais rico dos quatro — toggle de tipo, campo monetário com prefixo R\$, categoria e conta em `select` com skeleton enquanto carrega. Acessado de qualquer tela via atalho de teclado `N` .

tipo valor descrição categoria conta

### Nova conta

Instituição  
Tipo de conta

### Nova conta

Cadastro de conta bancária ou cartão. Campo de instituição é texto livre (ex: "Nubank, Itaú, BTG...") — não há autocomplete de bancos brasileiros ainda, apesar da dependência `@edusites/bancos-brasil` já estar instalada no projeto.

instituição tipo

### Novo aporte

Ativo (ticker)

### Novo aporte

Quantidade

Registro de compra de ativo por ticker ou nome livre (ex: IVVB11). Usado a partir da tela de Investimentos.

ativo

quantidade

Nova categoria

Ícone (grid)

Nome

### Nova categoria

Seletor de ícone visual + nome livre (ex: "Restaurantes, Pets, Viagens..."). É o único modal com escolha de ícone via grid clicável.

ícone

nome

Abertura é orquestrada por um composable único ( `useModal` ), que guarda qual dos 4 nomes está ativo — só um modal pode estar aberto por vez em toda a aplicação.

## 06 – feature destaque

# Kira — copiloto financeiro

Chat de IA dedicado (rota própria, não um widget flutuante), marcado como `beta`. Estado vazio traz avatar animado e 4 chips de sugestão que cobrem as perguntas mais prováveis de um usuário novo:

"Qual é meu saldo atual?"

"Onde gastei mais este mês?"

"Quanto tenho em investimentos?"

"Estou dentro do orçamento?"

Visualmente diferenciada do resto do produto pelo tom roxo ( `--pur #7C5CF5` ) — único lugar da interface que usa essa cor, o que a torna reconhecível como "zona de IA" em qualquer tela.

## 07 – onde estamos

# Estado atual

**FEITO** 11 telas do app + landing Quebec funcionando em Nuxt 4

**FEITO** Backend Rails 8 com 7 controllers de API, auth JWT, 6 migrations aplicadas

**FEITO** Padrão de modal único (drawer) consolidado nas 4 telas de criação

**FEITO** Deploy da API reconfigurado para Render (Kamal), substituindo tentativa via Vercel/Railway

**ABERTO** `/configuracoes` sem entrada de navegação — decidir onde expor

**ABERTO** Sem tema claro implementado — paleta atual é dark-only

**ABERTO** Autocomplete de bancos brasileiros (dependência já instalada, não usada no modal de conta)

**ATENÇÃO** Stack legado (`Streamlit:` `app.py` , `pages/` , `core/` , `web/` Next.js) ainda presente no working tree, marcado para remoção — não usar como referência de UI atual

Klipper · monorepo `apps/klipper-web` + `apps/klipper-api` + `apps/quebec-web` · gerado a partir do estado do repositório em 2026-07-06